8.2.1 Rego policies example

Here's a straightforward Rego policy that limits access to confidential documents based on department membership, similar to the earlier XACML example.

```
package acme.authz

import rego.v1

default allow := false

allow if {
  input.resource.confidential == true
  input.principal.department == input.resource.owner.department
}
```

This policy returns `true` (i.e., allows access) only if the document is marked as confidential and the requesting principal is in the same department as the owner.

The equivalent Cedar policy (from above) expresses the same logic:

```
permit(principal, action, resource)
when {
    resource.confidential == true &&
    principal.department == resource.owner.department
};
```

In this example, the logic is almost the same in both languages because we're not leveraging Cedar's scoping features; the access decision is fully determined by the condition block.

8.2.2 Comparing Rego with Cedar

Recall that Cedar allows you to incorporate constraints on the principal, action, and resource in the policy's scope, making them an integral part of the policy's structure rather than just its logic. For example, the following Cedar policy restricts delete actions to only principals in the admin group.